

Data Structure and Algorithm

(PMSCS 623P)

Lecture 4

LinearArrays

- A linear array is a list of a finite number of n homogeneous data elements (that is, data elements of the same type) such that:
 - The elements of the array are referenced respectively by an index set consisting of n consecutive numbers.
 - The elements of the array are stored respectively in successive memory locations.

Linear Arrays

- The number n of elements is called the length or size of the array.
- The index set consists of the integers $0, 1, 2, \dots, n-1$.
- Length or the number of data elements of the array can be obtained from the index set by

$$\text{Length} = \text{UB} - \text{LB} + 1$$

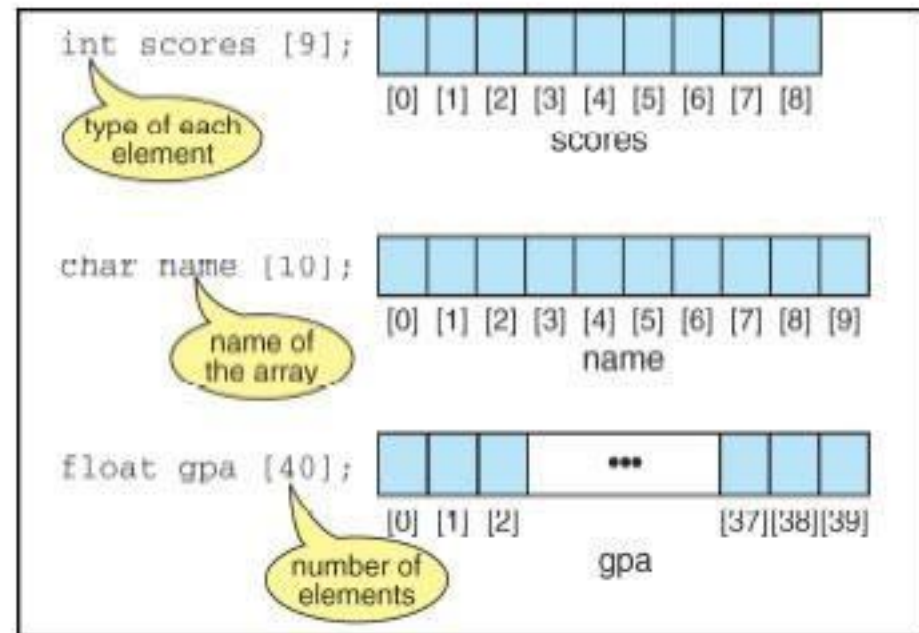
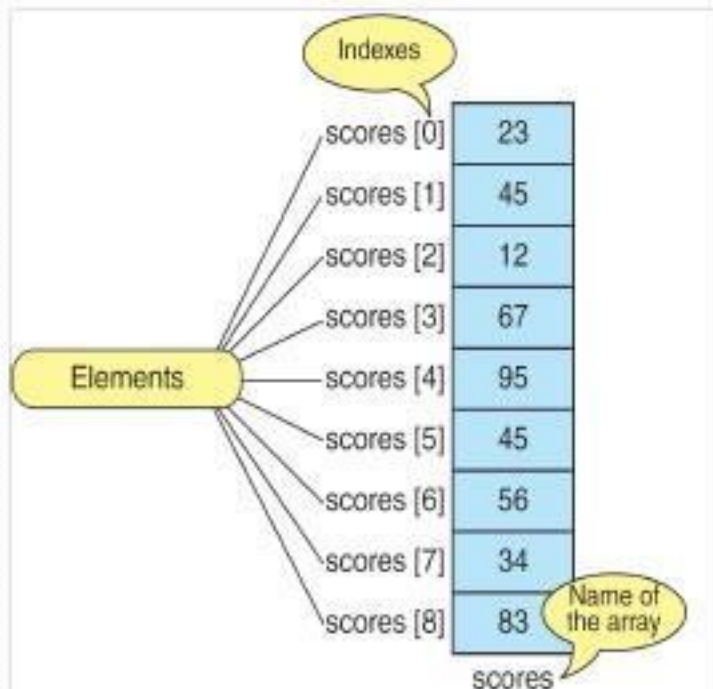
where

UB is the largest index called the upper bound and

LB is the smallest index called the lower bound of the arrays

Linear Arrays

- Element of an array **A** may be denoted by
 - Subscript notation **$A_1, A_2, \dots, A_n$**
 - Parentheses notation $A(1), A(2), \dots, A(n)$
 - Bracket notation $A[1], A[2], \dots, A[n]$
- **The number K in $A[K]$ is called subscript or an index and $A[K]$ is called a subscripted variable.**



Declaring and Initializing Arrays

Initializing Arrays

Initialization of an array either one by one or using a single statement as follows –

```
int ar[5];
```

```
ar[0]=10; ar[1]=20;
```

```
double balance[5] = {1000.0, 2.0, 3.4, 7.0, 50.0};
```

Representation of Linear Array in Memory

Actual Address of the 1st
element of the array is known as

Base Address (B)

Here it is 1100



Memory space acquired by every
element in the Array is called

Width (W)

Here it is 4 bytes

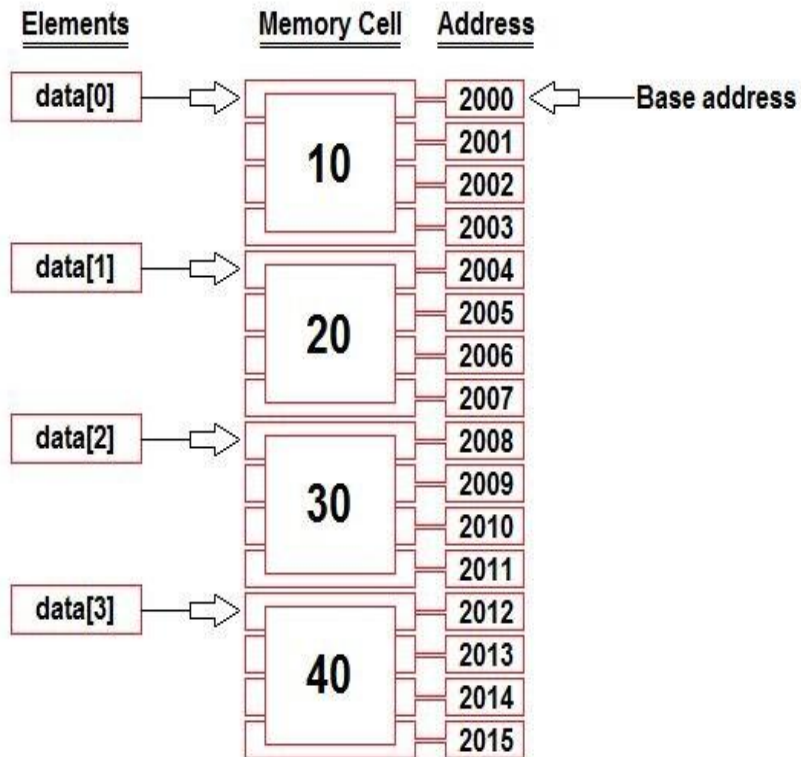


Actual Address in the Memory	1100	1104	1108	1112	1116	1120
Elements	15	7	11	44	93	20
Address with respect to the Array (Subscript)	0	1	2	3	4	5



Lower Limit/Bound
of Subscript (**LB**)

Representation of Linear Array in Memory



arr[0]	arr[1]	arr[2]	arr[3]	arr[4]
10	20	30	40	50
2000	2004	2008	2012	2016

Representation of Linear Array in Memory

Address of the memory				indexes of array elements
	1001	20	1	
	1002			
	1003	50	2	
	1004			
	1005	102	3	
	1006			
	1007	600	4	
	1008			
	1009	2	5	
	1010			
	1011	34	6	
	1012			
	1013	500	7	
	1014			
	1015	100	8	
	1016			

Memory Location									
200	201	202	203	204	205	206	▪	▪	▪
U	B	F	D	A	E	C	▪	▪	▪
0	1	2	3	4	5	6	▪	▪	▪
Index									

Example 1

Find the address for LA[6] Each element of the array occupy 1 byte

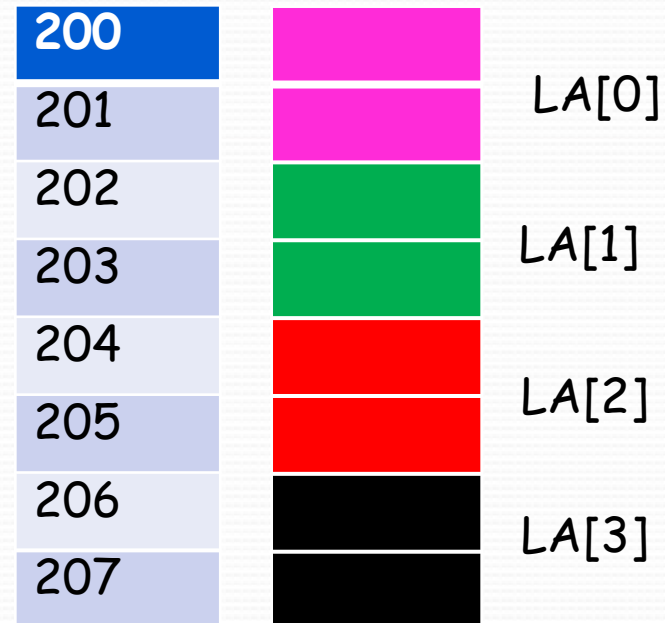
200		LA[1]
201		LA[2]
202		LA[3]
203		LA[4]
204		LA[5]
205		LA[6]
206		LA[7]
207		LA[8]

$$LOC(LA[K]) = \text{Base}(LA) + w(K - \text{lower bound})$$

$$LOC(LA[6]) = 200 + 1(6 - 1) = 205$$

Example 2

Find the address for LA[16] Each element of the array occupy 2 byte



$$\text{LOC}(\text{LA}[\text{K}]) = \text{Base}(\text{LA}) + w(\text{K} - \text{lower bound}) :$$

$$\text{LOC}(\text{LA}[16]) = 200 + 2(16 - 0) = 232$$



Data Structures

□ Operations on Linear Array

- **Traversing:** means to visit all the elements of the array in an operation is called traversing.
- **Insertion:** means to put values into an array.
- **Deletion / Remove:** to delete a value from an array.
- **Sorting:** Re-arrangement of values in an array in a specific order (Ascending / Descending) is called sorting.
- **Searching:** The process of finding the location of a particular element in an array is called searching.
- There are two popular searching techniques/mechanisms: **Linear search** and **binary search**.

The Search Problem

- Given an array A storing n numbers, and a target number v , locate the position in A (if it exists) where $A[i] = v$
- **Example**
 - Input: $A = \{3, 8, 2, 15, 99, 52\}$, $v = 99$
 - Output: position 4
- **Notes**
 - Array positions (indexes) go from $0..n-1$
 - When the target does not exist in the array, return an undefined position, such as -1

Algorithm 1: Linear Search

Algorithm LinearSearch(A, v) :

Input: An array A of n numbers, search target v

Output: position i where $A[i]=v$, -1 if not found

```
for  $i \leftarrow 0$  to  $n - 1$  do
    if  $A[i] = v$  then
        return  $i$ 
return  $-1$ 
```


Linear Search time complexity

- **Worst-case:** $O(n)$
 - If v does not exist or if v is the last element in the array
- **Best-case:** $O(1)$
 - When the target element v is the first element in the array
- **Average-case:** $O(n)$
 - e.g., if the target element is somewhere in the middle of the array, the for-loop will iterate $n/2$ times. Still $O(n)$

Binary Search (recursive)

// Returns location of x in the sorted array A[first..last] if x is in A, otherwise returns -1

Algorithm BinarySearch(A, first, last, x)

if last $\geq$ first **then**

 mid = first + (last - first)/2

 // If the element is present at the middle itself

if A[mid] = x **then**

return mid

 // If element is smaller than mid, then it can only be present in left sub-array

else if A[mid] > x **then**

return BinarySearch(A, first, mid-1, x)

 // Otherwise the element can only be present in the right sub-array

else

return BinarySearch(A, mid+1, last, x);

return -1 // We reach here when element is not present in A

Binary Search (iteration)

Binary Search

Pseudocode (using iteration)

```
BinarySearch(list[], min, max, key)
while min ≤ max do
    mid = (max+min) / 2
    if list[mid] > key then
        max = mid-1
    else if list[mid] < key then
        min = mid+1
    else
        return mid
    end if
end while
return false
```

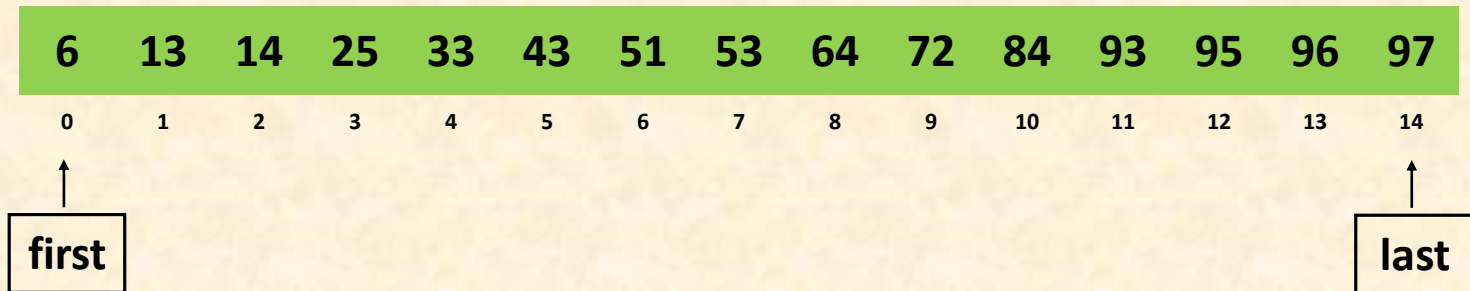
Retrieving an Item from Sorted List

- Find 84

6	13	14	25	33	43	51	53	64	72	84	93	95	96	97
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Retrieving an Item from Sorted List

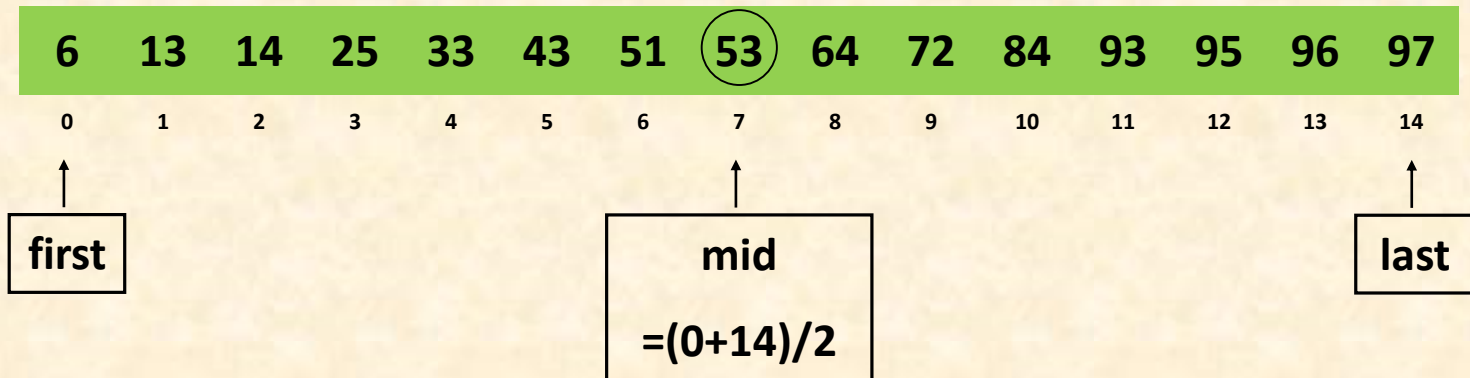
- Find 84



- Step 1

Retrieving an Item from Sorted List

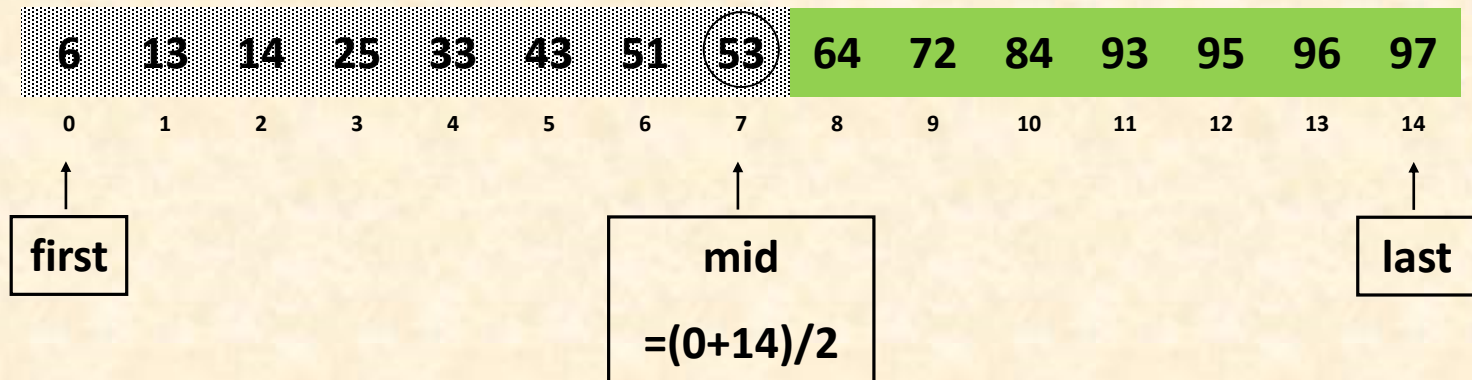
- Find 84



- Step 1

Retrieving an Item from Sorted List

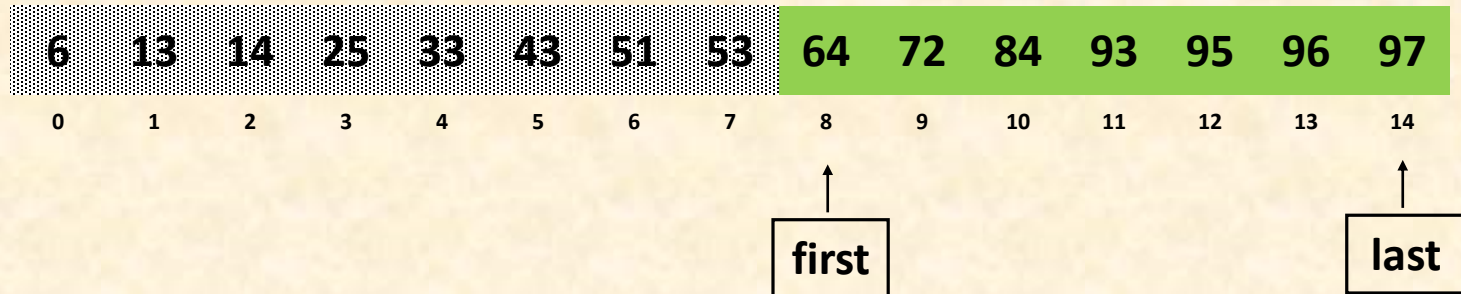
- Find 84



- Step 1

Retrieving an Item from Sorted List

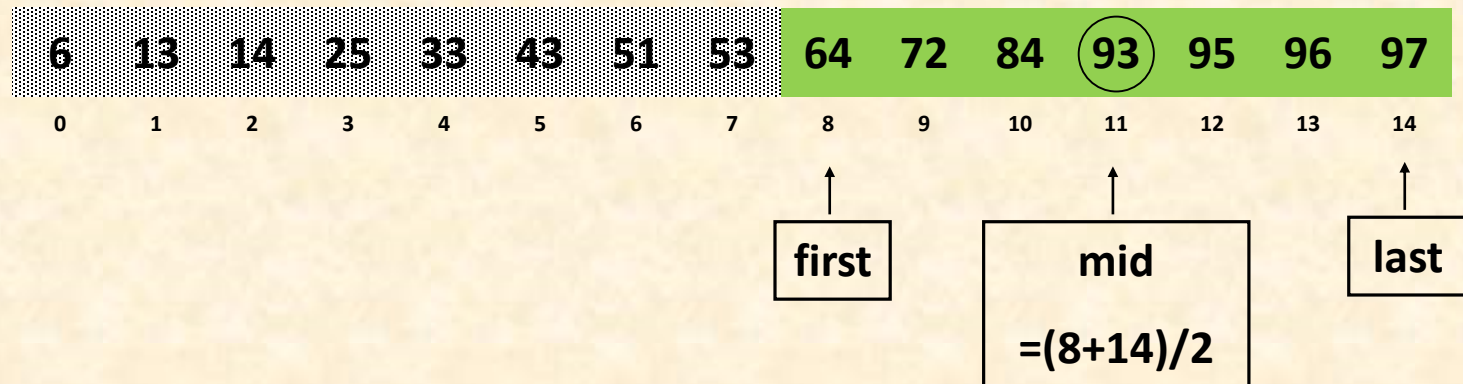
- Find 84



- Step 2

Retrieving an Item from Sorted List

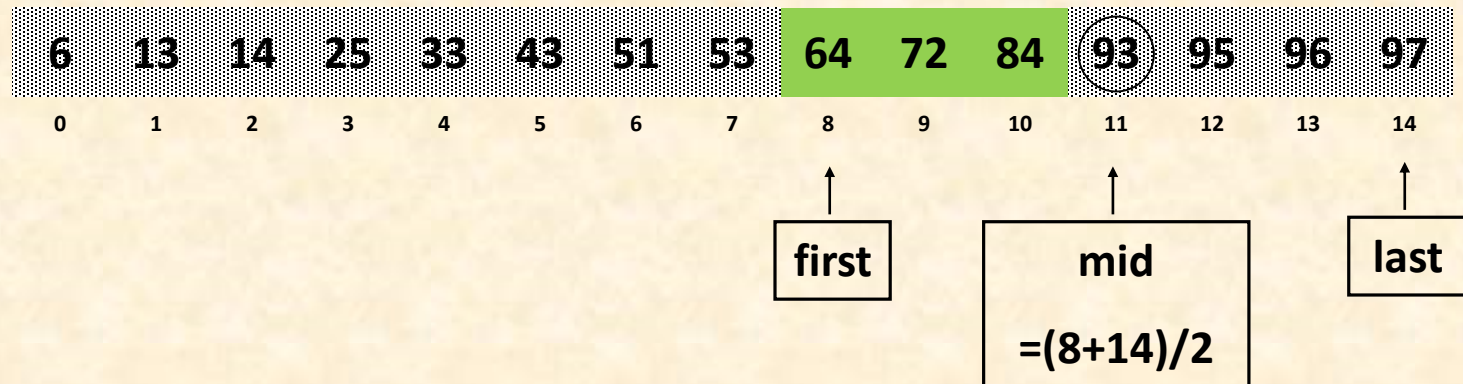
- Find 84



- Step 2

Retrieving an Item from Sorted List

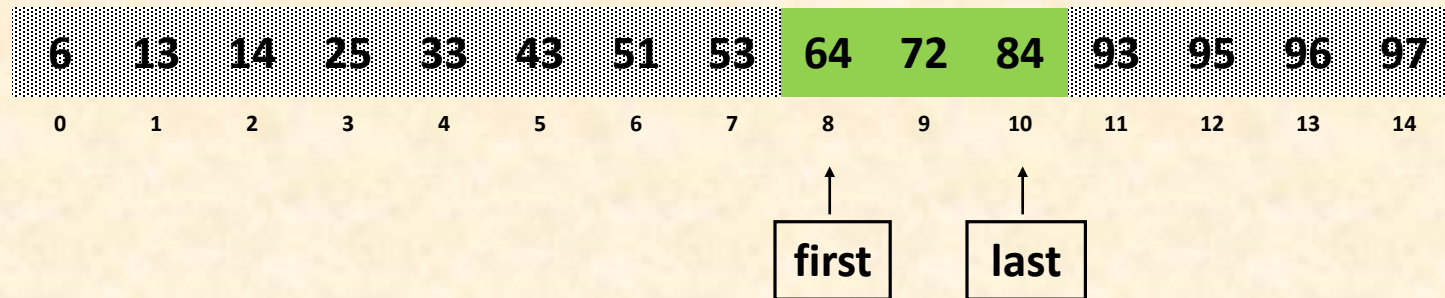
- Find 84



- Step 2

Retrieving an Item from Sorted List

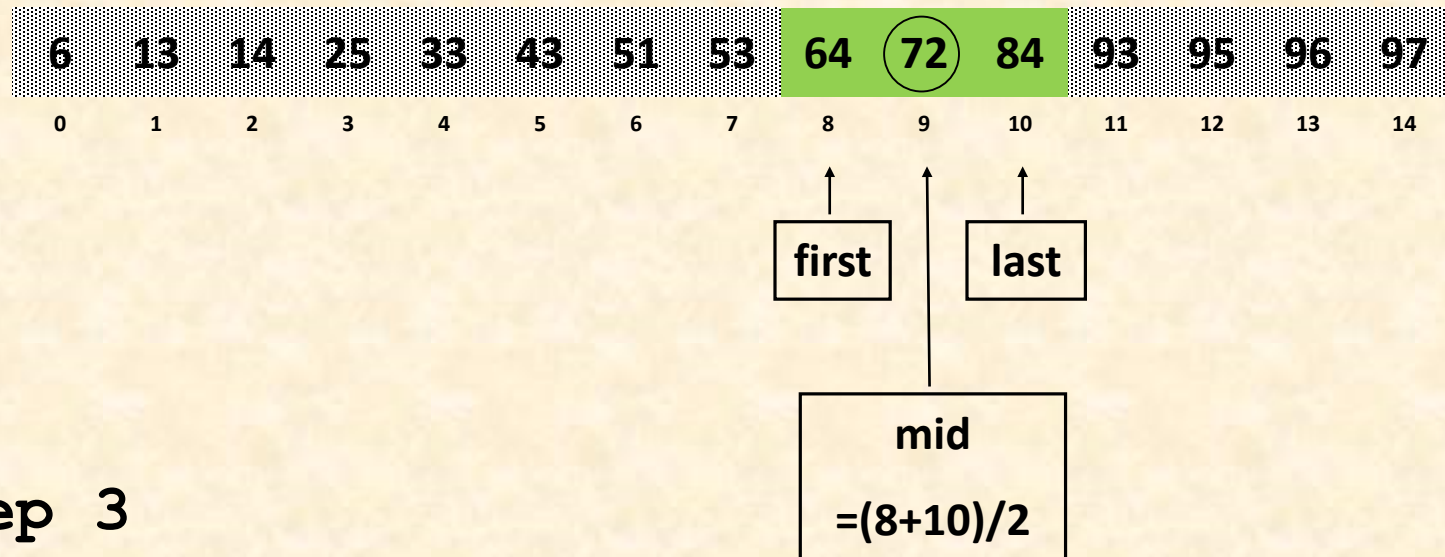
- Find 84



- Step 3

Retrieving an Item from Sorted List

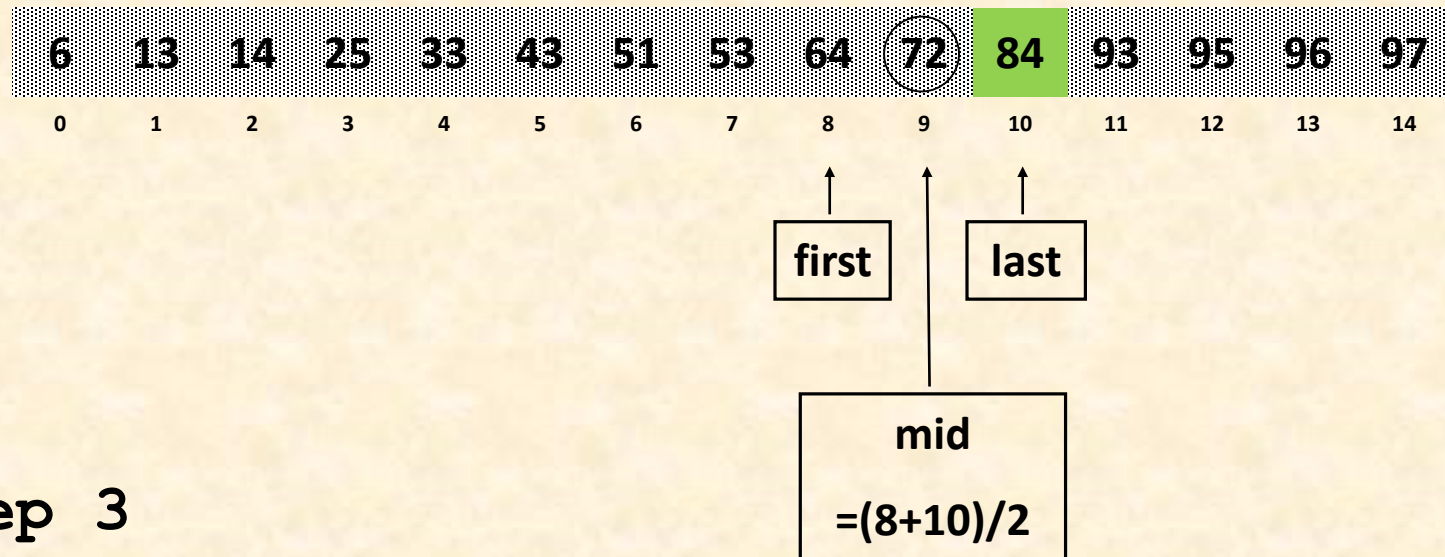
- Find 84



- Step 3

Retrieving an Item from Sorted List

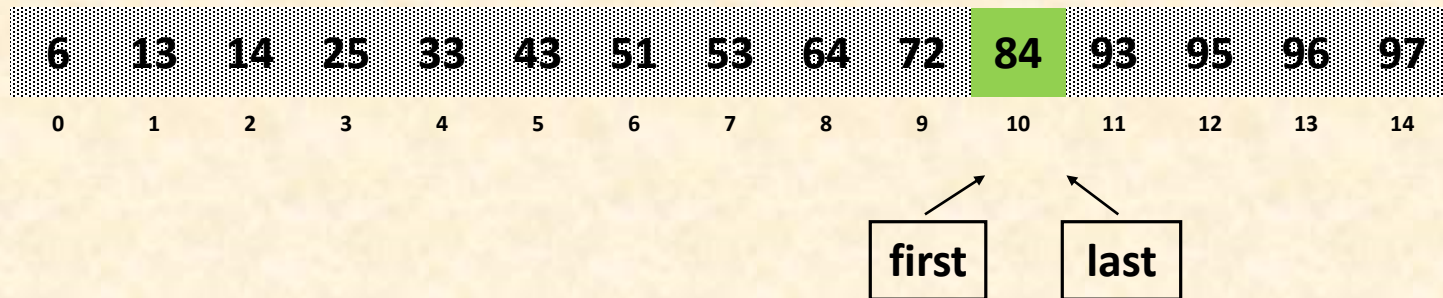
- Find 84



- Step 3

Retrieving an Item from Sorted List

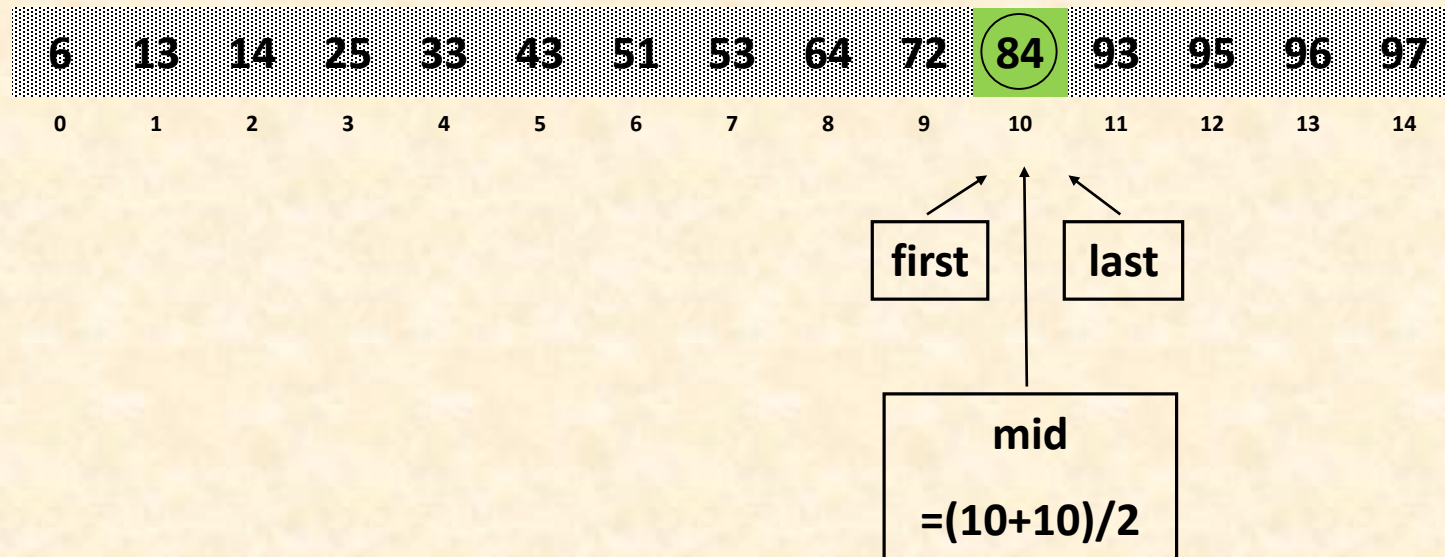
- Find 84



- Step 4

Retrieving an Item from Sorted List

- Find 84



- Step 4
- 84 found at the midpoint

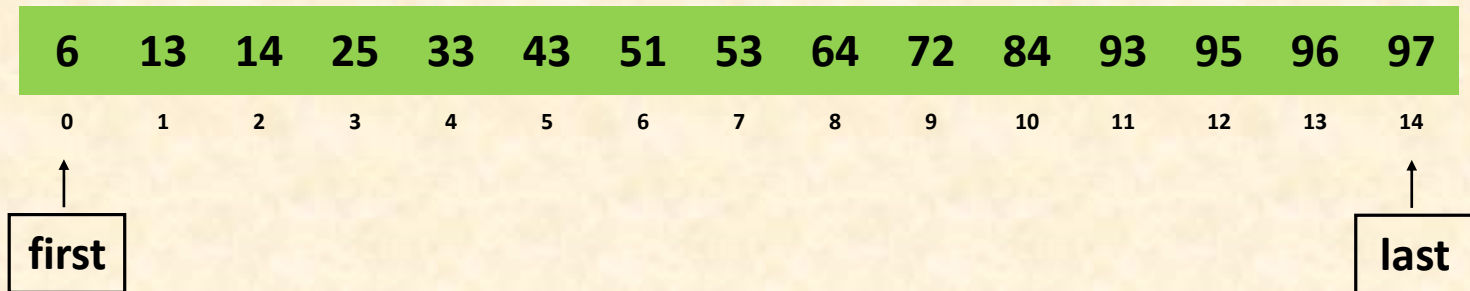
Retrieving an Item from Sorted List

- Find 73

6	13	14	25	33	43	51	53	64	72	84	93	95	96	97
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Retrieving an Item from Sorted List

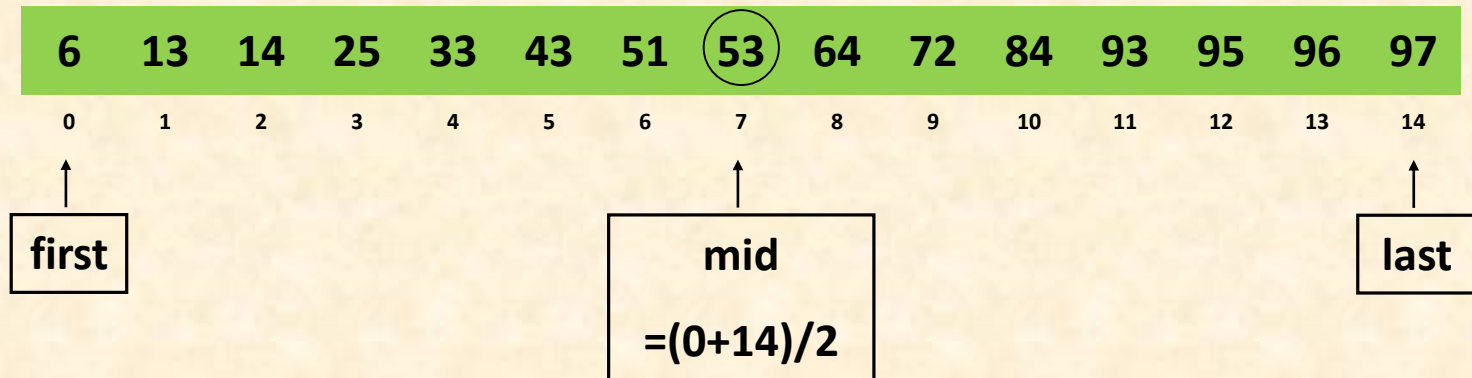
- Find 73



- Step 1

Retrieving an Item from Sorted List

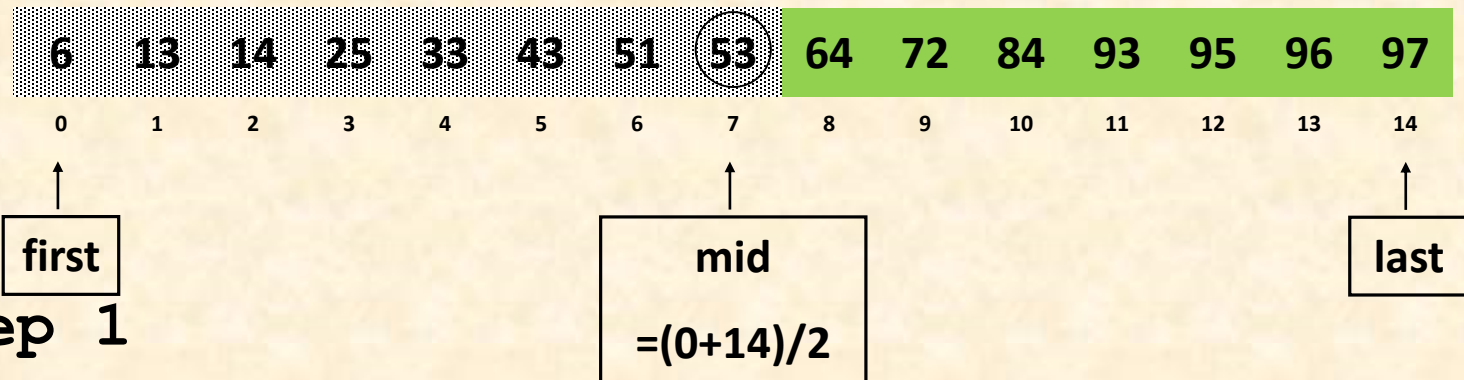
- Find 73



- Step 1

Retrieving an Item from Sorted List

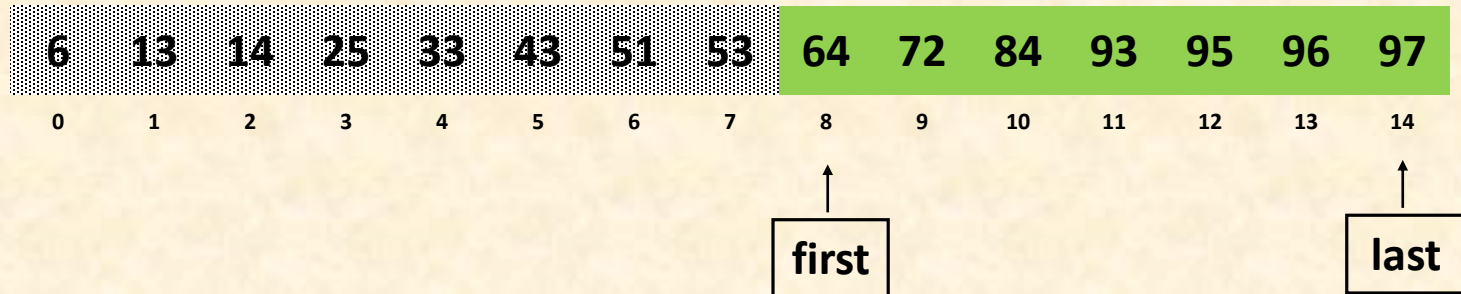
- Find 73



- Step 1

Retrieving an Item from Sorted List

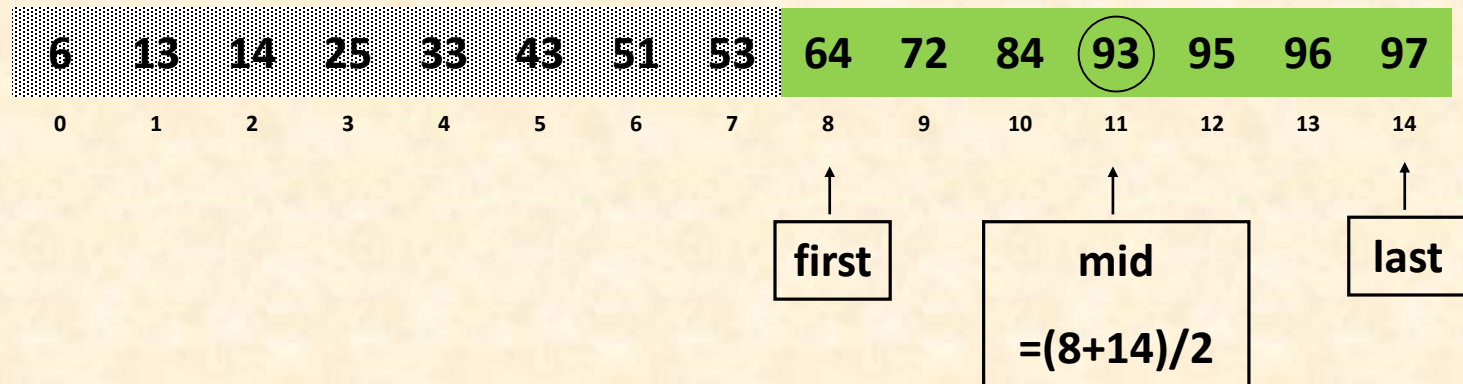
- Find 73



- Step 2

Retrieving an Item from Sorted List

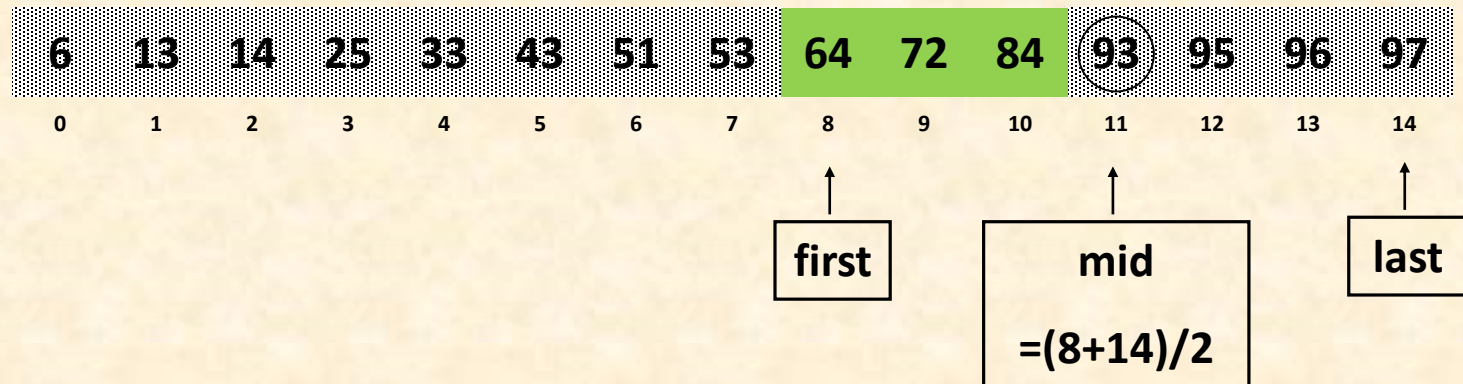
- Find 73



- Step 2

Retrieving an Item from Sorted List

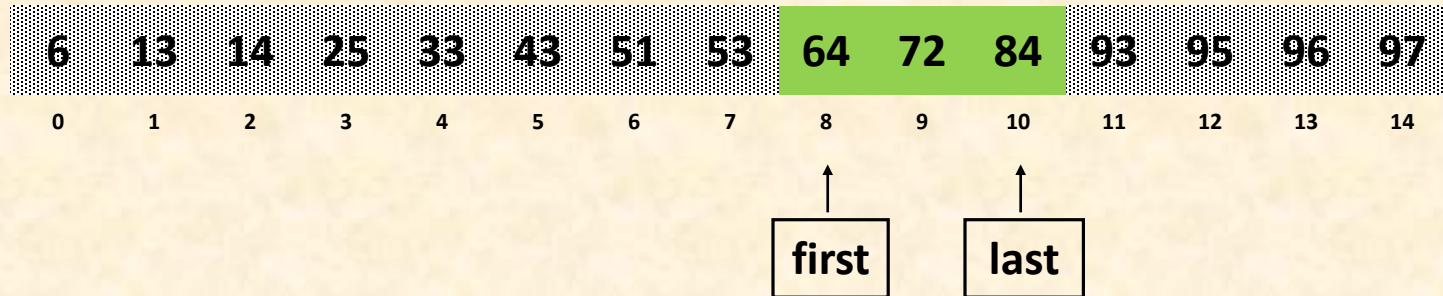
- Find 73



- Step 2

Retrieving an Item from Sorted List

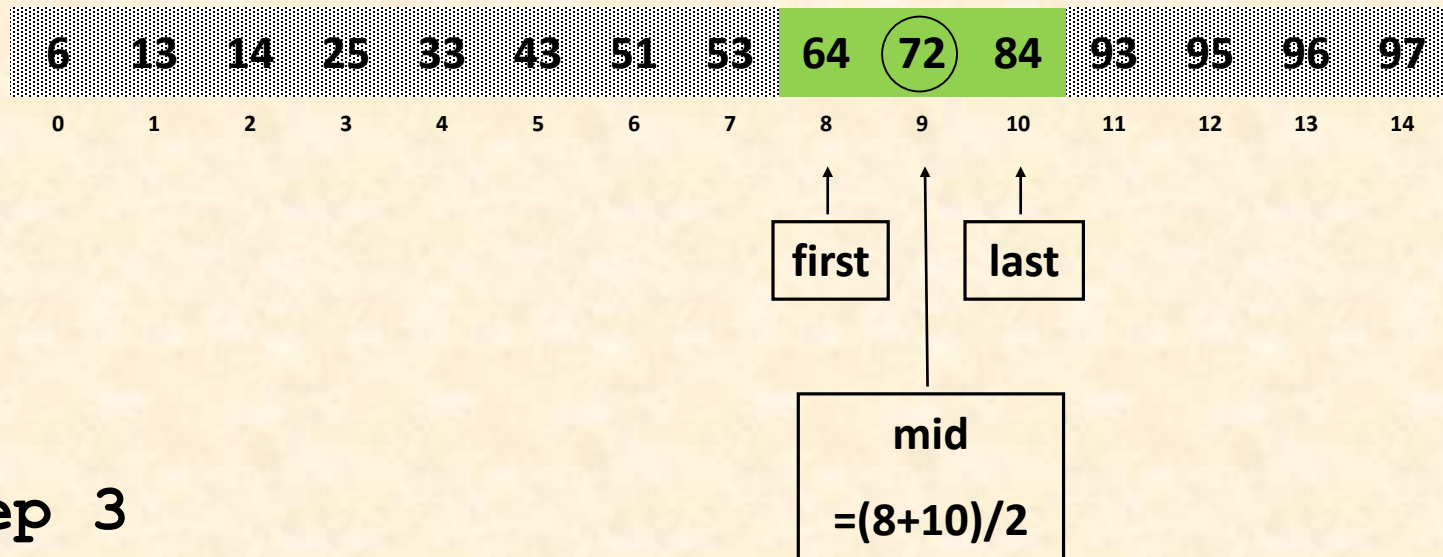
- Find 73



- Step 3

Retrieving an Item from Sorted List

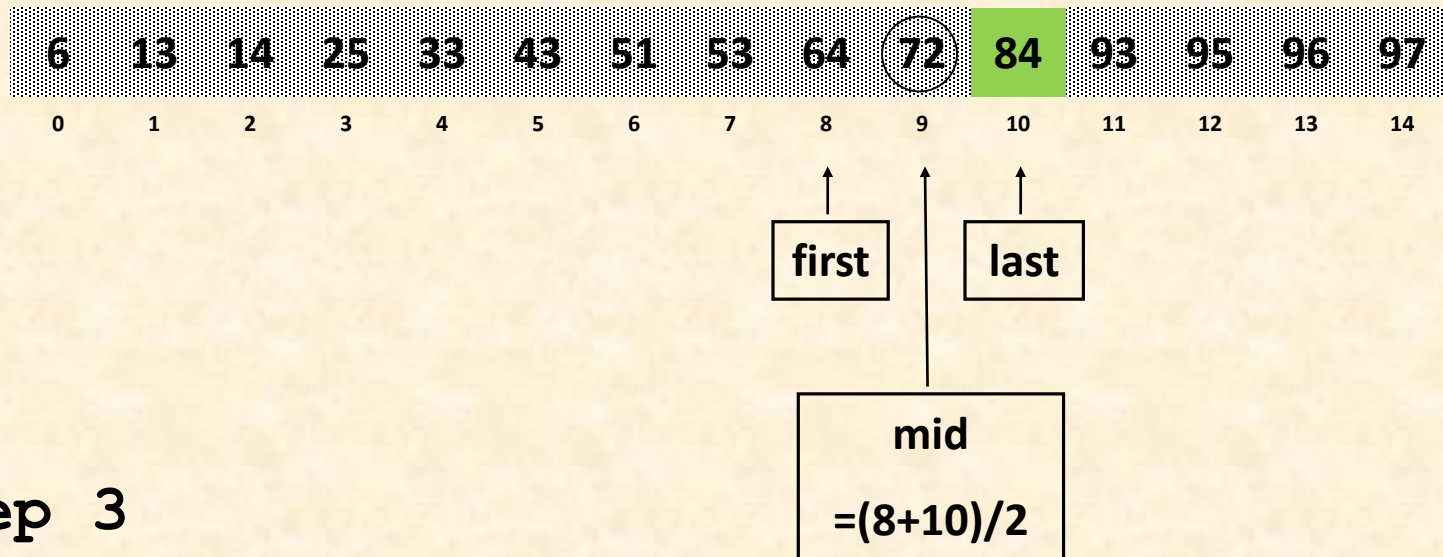
- Find 73



- Step 3

Retrieving an Item from Sorted List

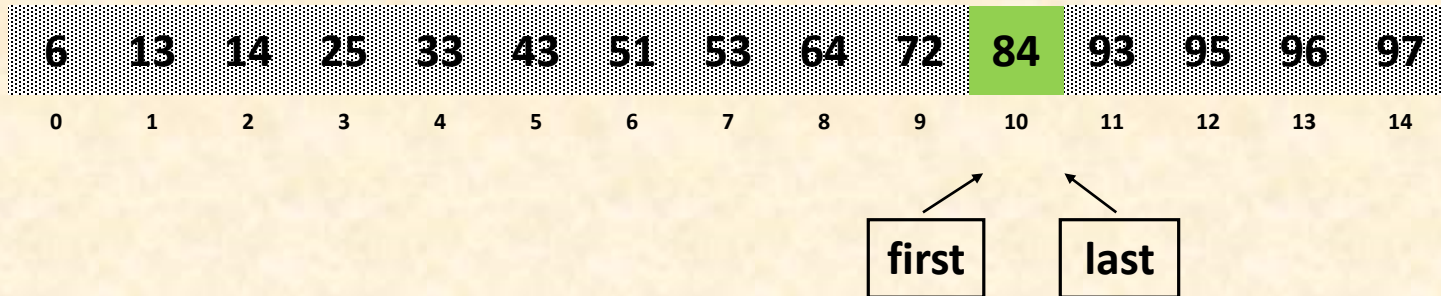
- Find 73



- Step 3

Retrieving an Item from Sorted List

- Find 73



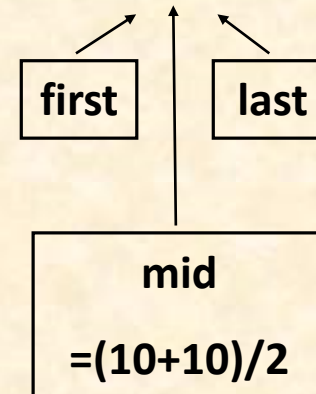
- Step 4

Retrieving an Item from Sorted List

- Find 73

6	13	14	25	33	43	51	53	64	72	84	93	95	96	97
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

- Step 4

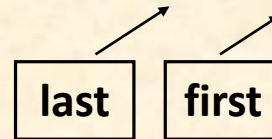


Retrieving an Item from Sorted List

- Find 73

6	13	14	25	33	43	51	53	64	72	84	93	95	96	97
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

- Step 5



- Item $\neq$ info[mid] (indicates the absence of the item)

Binary Search (recursive)

// Returns location of x in the sorted array A[first..last] if x is in A, otherwise returns -1

Algorithm BinarySearch(A, first, last, x)

if last $\geq$ first **then**

 mid = first + (last - first)/2

 // If the element is present at the middle itself

if A[mid] = x **then**

return mid

 // If element is smaller than mid, then it can only be present in left sub-array

else if A[mid] > x **then**

return BinarySearch(A, first, mid-1, x)

 // Otherwise the element can only be present in the right sub-array

else

return BinarySearch(A, mid+1, last, x);

return -1 // We reach here when element is not present in A

Time complexity analysis

- Examining the algorithm yields the following observations:
 - ❑ The non-recursive work takes constant time C .
 - ❑ The single recursive call has input half the size of the original.
 - ❑ The algorithm terminates when the array has size no larger than one.

Analysis: solving recurrence

$$T(n) = T(n/2) + O(1), \text{ and } O(1)=c$$

$$T(n) = T\left(\frac{n}{2}\right) + c$$

$$= \left(T\left(\frac{n}{4}\right) + c\right) + c$$

$$= T\left(\frac{n}{4}\right) + 2c$$

$$= \left(T\left(\frac{n}{8}\right) + c\right) + 2c$$

$$= T\left(\frac{n}{8}\right) + 3c$$

$$= T\left(\frac{n}{2^k}\right) + kc$$

Since $n=2^k$, we have $k=\log_2 n$

$$T(n) = T\left(\frac{n}{2^k}\right) + kc$$

$$= T(1) + (\log n)c$$

$$= O(\log n)$$

Time complexity analysis

To analyze the binary search algorithm, we need to recall that from consideration. **each comparison eliminates about half of the remaining items**

What is the maximum number of comparisons this algorithm will require to check the entire list?

If we start with n items, about $n/2$ items will be left after the first comparison. After the second comparison, there will be about $n/4$. Then $n/8$, $n/16$, and so on.

So the worst case would be

$$[N]/2 + [(N/2)]/2 + [((N/2)/2)]/2.....$$

i.e:

$$N/2^1 + N/2^2 + N/2^3 + + N/2^x$$

That shows the worst case is when you reach $N/2^x$ where x is such that $2^x = N$. Now since mathematically worst case is when the value of

$$2^x = N$$

$$\Rightarrow \log_2(2^x) = \log_2(N)$$

$$\Rightarrow x * \log_2(2) = \log_2(N)$$

$$\Rightarrow x * 1 = \log_2(N)$$



Data Structures

❑ Complexity of the Binary Search Algorithm

- The **Complexity** of the **Binary Search** algorithm is measured by the maximum (worst case) number of Comparisons it performs for searching operations. The searched array is divided by 2 for each comparison/iteration. Therefore, we require at most $f(n)$ comparisons to locate ITEM:

$$2f(n) > n \text{ or equivalently, } f(n) = \lceil \log_2 n \rceil + 1$$

- Approximately, equal to $\log_2 n$.

Example:

- If a given sorted array contains 1024 elements, then the maximum number of comparisons required is:

$$\log_2(1024) = 10 \text{ (only 10 comparisons are enough)}$$



Data Structures

❑ Limitations of the Binary Search Algorithm

The algorithm requires two conditions:

- The list must be sorted.
- One must have direct access to the middle element in any sub list.



Data Structures

❑ Insertion into Arrays

Inserting an element somewhere in the middle of the array require that each subsequent element be moved downward to new locations to accommodate the new element and **keep the order of the other elements**.

Algorithm 4.2: (Inserting into a linear array)

Here LA is linear array with N elements and K is a positive integer such that $K \leq N$. This algorithm inserts an element ITEM into the K-th position in LA.

- Step 1. Set $J := N$
- Step 2. Repeat Steps 3 and 4 while $J \geq K$
- Step 3. Set $LA[J+1] := LA[J]$
- Step 4. Set $J := J - 1$
- [End of step 2 loop]
- Step 5. Set $LA[K] := \text{ITEM}$
- Step 6. Set $N := N + 1$
- Step 7. Exit



Data Structures

❑ Deletion of elements from linear arrays

Deleting an element somewhere in the middle of the array requires that each subsequent element be moved one location upward in order to “fill up” the array.

Algorithm 4.3: (Deleting from a linear array)

Here LA is a linear array with N elements and K is a positive integer such that $K \leq N$. This algorithm deletes the K-th element from LA.

Step 1. Set ITEM: = LA [K]

Step 2. Repeat for J=K to N-1

Set LA [J]: =LA [J+1]

[End of loop]

Step 3. Set N:=N-1

Step 4. Exit

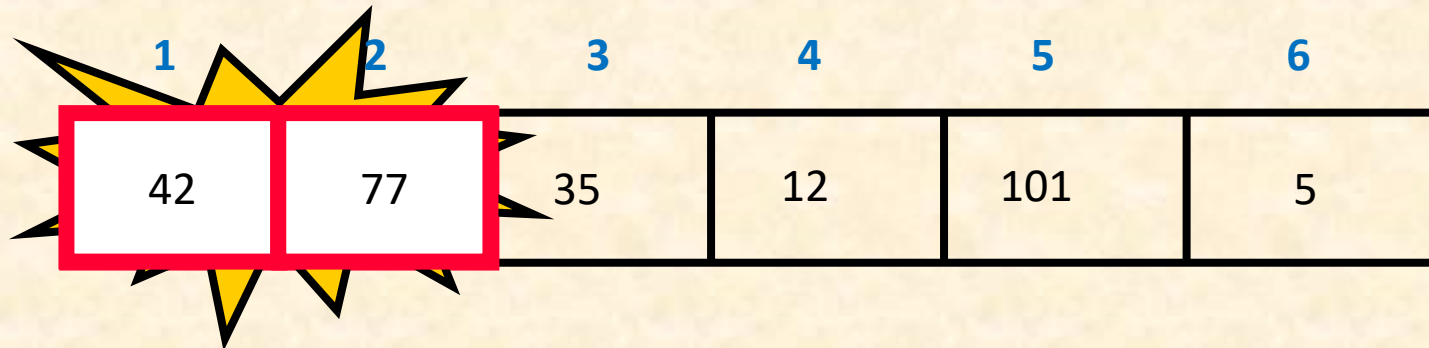
"Bubbling Up" the Largest Element

- **Traverse a collection of elements**
 - Move from the front to the end
 - “Bubble” the **largest value** to the end using **pair-wise comparisons and swapping**

1	2	3	4	5	6
77	42	35	12	101	5

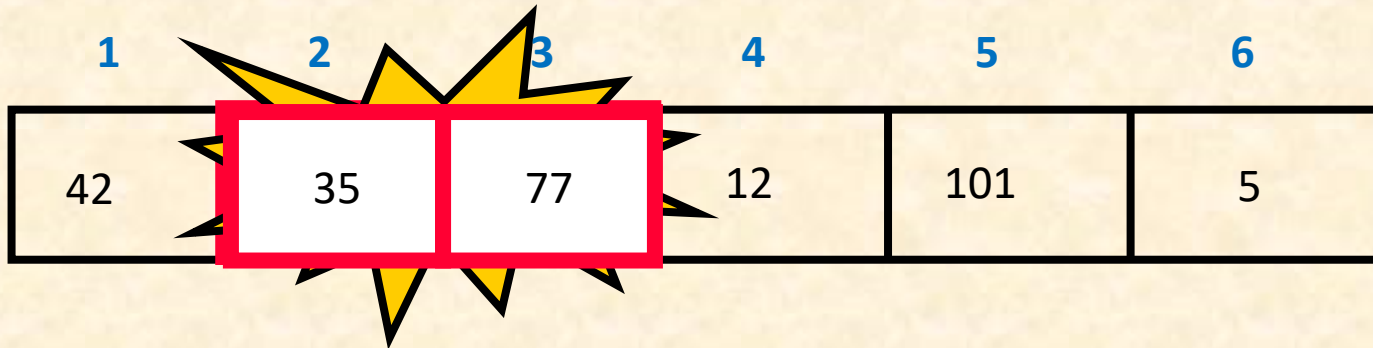
"Bubbling Up" the Largest Element

- **Traverse a collection of elements**
 - Move from the front to the end
 - “Bubble” the largest value to the end using pairwise comparisons and swapping



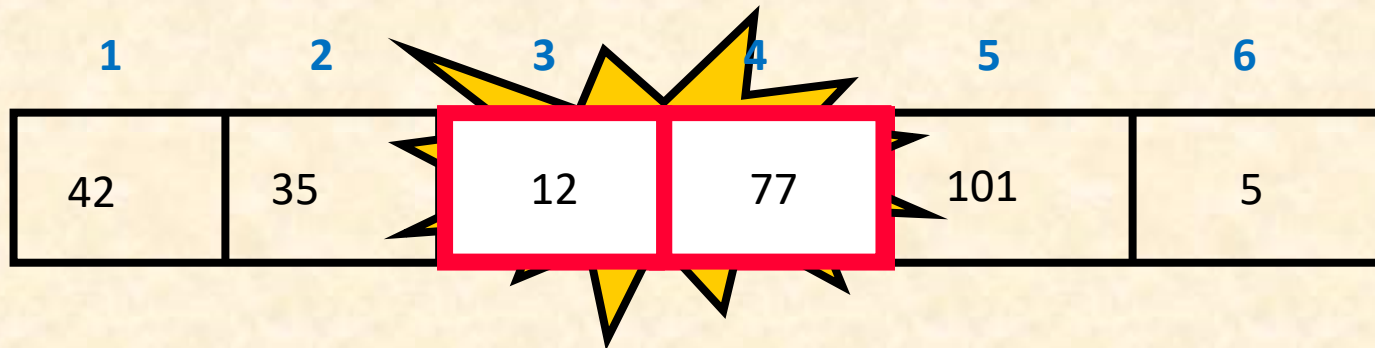
"Bubbling Up" the Largest Element

- **Traverse a collection of elements**
 - Move from the front to the end
 - “Bubble” the largest value to the end using pairwise comparisons and swapping



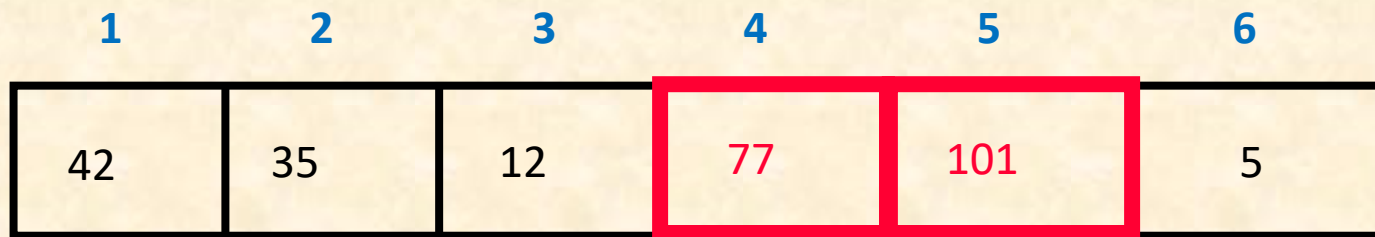
"Bubbling Up" the Largest Element

- **Traverse a collection of elements**
 - Move from the front to the end
 - “Bubble” the largest value to the end using pairwise comparisons and swapping



"Bubbling Up" the Largest Element

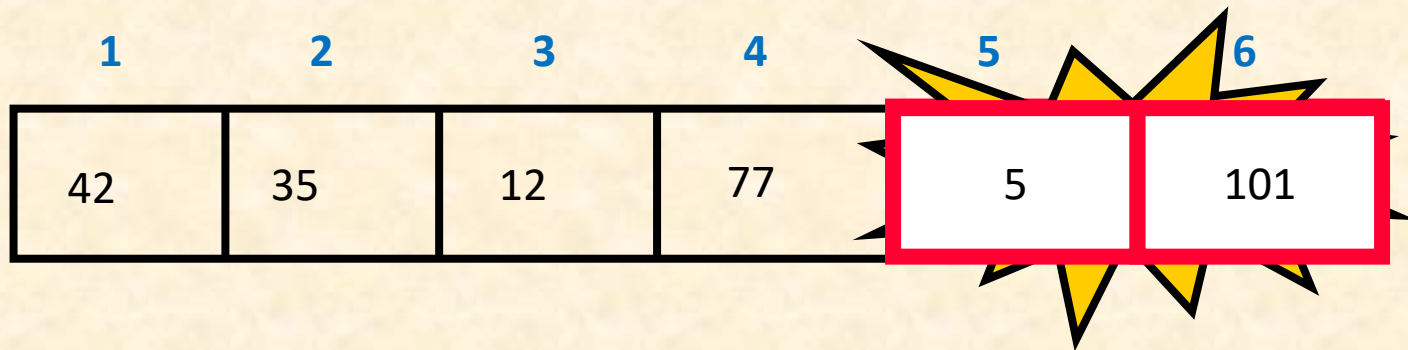
- **Traverse a collection of elements**
 - Move from the front to the end
 - “Bubble” the largest value to the end using pairwise comparisons and swapping



No need to swap

"Bubbling Up" the Largest Element

- **Traverse a collection of elements**
 - Move from the front to the end
 - “Bubble” the largest value to the end using pairwise comparisons and swapping



"Bubbling Up" the Largest Element

- **Traverse a collection of elements**
 - Move from the front to the end
 - “Bubble” the largest value to the end using pairwise comparisons and swapping

1	2	3	4	5	6
42	35	12	77	5	101

Largest value correctly placed

Bubble Sort

16 2	16 2	22	18 2	18 7	22
6	12	18 4	18 4	17	22

- Given n numbers to sort:
- Repeat the following n-1 times:
 - For each **pair** of adjacent numbers:
 - If the number on the left is greater than the number on the right, swap them.

Bubble Sort

6	18	18	14	17	22
---	---------------	---------------	----	----	----

6	8	12	14	17	22
---	---	----	----	----	----

- Given n numbers to sort:
- Repeat the following n-1 times:
 - For each **pair** of adjacent numbers:
 - If the number on the left is greater than the number on the right, swap them.

Bubble Sort

- Given n numbers to sort:
 - Repeat the following $n-1$ times:
 - For each pair of adjacent numbers:
 - If the number on the left is greater than the number on the right, swap them
-
- How efficient is bubble sort?
 - In general, given n numbers to sort, it performs n^2 comparisons
 - The same as selection sort
 - Is there a simple way to improve on the basic bubble sort?
 - Yes! Stop after going through without making any swaps
 - This will only help some of the time

Bubble Sort

- Given n numbers to sort:
- Repeat the following $n-1$ times:
 - For each pair of adjacent numbers:
 - If the number on the left is greater than the number on the right, swap them

Try one!

15	3	11	19	4	7
----	---	----	----	---	---



Data Structures

□ Bubble Sort

Suppose the following numbers are stored in an array A:

A: 32, 51, 27, 85, 66, 23, 13, 57

Apply the bubble sort to the array A. Show each pass separately.

Pass 1: (a) Compare A_1 and A_2 . Since $32 < 51$, the list is not altered.

A: 32, 51, 27, 85, 66, 23, 13, 57

(b) Compare A_2 and A_3 . Since $51 > 27$, interchange 51 and 27.

A: 32, 27, 51, 85, 66, 23, 13, 57

(c) Compare A_3 and A_4 . Since $51 < 85$, the list is not altered.

A: 32, 27, 51, 85, 66, 23, 13, 57

(d) Compare A_4 and A_5 . Since $85 > 66$, interchange 85 and 66.

A: 32, 27, 51, 66, 85, 23, 13, 57



Data Structures

❑ Bubble Sort

Suppose the following numbers are stored in an array A:

A: 32, 51, 27, 85, 66, 23, 13, 57

Apply the bubble sort to the array A. Show each pass separately.

Pass 1: (e) Compare A_5 and A_6 . Since $85 > 23$, interchange 85 and 23.

A: 32, 27, 51, 66, 23, 85, 13, 57

(f) Compare A_6 and A_7 . Since $85 > 13$, interchange 85 and 13.

A: 32, 27, 51, 66, 23, 13, 85, 57

(g) Compare A_7 and A_8 . Since $85 > 57$, interchange 85 and 57.

A: 32, 27, 51, 66, 23, 13, 57, 85



Data Structures

❑ Bubble Sort

Suppose the following numbers are stored in an array A:

A: 32, 51, 27, 85, 66, 23, 13, 57

Apply the bubble sort to the array A. Show each pass separately.

After Pass 1: A: 32, 27, 51, 66, 23, 13, 57, 85

Pass 2: 27, 32, 51, 66, 23, 13, 57, 85

27, 32, 51, 66, 23, 13, 57, 85

27, 32, 51, 66, 23, 13, 57, 85

27, 32, 51, 23, 66, 13, 57, 85

27, 32, 51, 23, 13, 66, 57, 85

27, 32, 51, 23, 13, 57, 66, 85



Data Structures

❑ Bubble Sort

Suppose the following numbers are stored in an array A:

A: 32, 51, 27, 85, 66, 23, 13, 57

Apply the bubble sort to the array A. Show each pass separately.

After Pass 1: A: 32, 27, 51, 66, 23, 13, 57, 85

After Pass 2: A: 27, 32, 51, 23, 13, 57, 66, 85

Pass 3: 27, 32, 51, 23, 13, 57, 66, 85

27, 32, 51, 23, 13, 57, 66, 85

27, 32, 23, 51, 13, 57, 66, 85

27, 32, 23, 13, 51, 57, 66, 85

27, 32, 23, 13, 51, 57, 66, 85



Data Structures

❑ Bubble Sort

Suppose the following numbers are stored in an array A:

A: 32, 51, 27, 85, 66, 23, 13, 57

Apply the bubble sort to the array A. Show each pass separately.

After Pass 1: A: 32, 27, 51, 66, 23, 13, 57, 85

After Pass 2: A: 27, 32, 51, 23, 13, 57, 66, 85

After Pass 3: A: 27, 32, 23, 13, 51, 57, 66, 85

Pass 4: 27, 32, 23, 13, 51, 57, 66, 85

27, 23, 32, 13, 51, 57, 66, 85

27, 23, 13, 32, 51, 57, 66, 85

27, 23, 13, 32, 51, 57, 66, 85



Data Structures

❑ Bubble Sort

Suppose the following numbers are stored in an array A:

A: 32, 51, 27, 85, 66, 23, 13, 57

Apply the bubble sort to the array A. Show each pass separately.

After Pass 1: A: 32, 27, 51, 66, 23, 13, 57, 85

After Pass 2: A: 27, 32, 51, 23, 13, 57, 66, 85

After Pass 3: A: 27, 32, 23, 13, 51, 57, 66, 85

After Pass 4: A: 27, 23, 13, 32, 51, 57, 66, 85

Pass 5: 23, 27, 13, 32, 51, 57, 66, 85

23, 13, 27, 32, 51, 57, 66, 85

23, 13, 27, 32, 51, 57, 66, 85



Data Structures

❑ Bubble Sort

Suppose the following numbers are stored in an array A:

A: 32, 51, 27, 85, 66, 23, 13, 57

Apply the bubble sort to the array A. Show each pass separately.

After Pass 1: A: 32, 27, 51, 66, 23, 13, 57, 85

After Pass 2: A: 27, 32, 51, 23, 13, 57, 66, 85

After Pass 3: A: 27, 32, 23, 13, 51, 57, 66, 85

After Pass 4: A: 27, 23, 13, 32, 51, 57, 66, 85

After Pass 5: A: 23, 13, 27, 32, 51, 57, 66, 85

Pass 6: 13, 23, 27, 32, 51, 57, 66, 85

13, 23, 27, 32, 51, 57, 66, 85



Data Structures

❑ Bubble Sort

Suppose the following numbers are stored in an array A:

A: 32, 51, 27, 85, 66, 23, 13, 57

Apply the bubble sort to the array A. Show each pass separately.

After Pass 1: A: 32, 27, 51, 66, 23, 13, 57, 85

After Pass 2: A: 27, 32, 51, 23, 13, 57, 66, 85

After Pass 3: A: 27, 32, 23, 13, 51, 57, 66, 85

After Pass 4: A: 27, 23, 13, 32, 51, 57, 66, 85

After Pass 5: A: 23, 13, 27, 32, 51, 57, 66, 85

After Pass 6: A: 13, 23, 27, 32, 51, 57, 66, 85

Pass 7: 13, 23, 27, 32, 51, 57, 66, 85



Data Structures

❑ Bubble Sort

Suppose the following numbers are stored in an array A:

A: 32, 51, 27, 85, 66, 23, 13, 57

Apply the bubble sort to the array A. Show each pass separately.

After Pass 1: A: 32, 27, 51, 66, 23, 13, 57, 85

After Pass 2: A: 27, 32, 51, 23, 13, 57, 66, 85

After Pass 3: A: 27, 32, 23, 13, 51, 57, 66, 85

After Pass 4: A: 27, 23, 13, 32, 51, 57, 66, 85

After Pass 5: A: 23, 13, 27, 32, 51, 57, 66, 85

After Pass 6: A: 13, 23, 27, 32, 51, 57, 66, 85

After Pass 7: A: 13, 23, 27, 32, 51, 57, 66, 85

Sorted List: A: 13, 23, 27, 32, 51, 57, 66, 85



Data Structures

❑ **Algorithm 4.4:** (Bubble Sort) BUBBLE (DATA, N)

Here DATA is an Array with N elements. This algorithm sorts the elements in DATA.

1. Repeat Steps 2 and 3 for $K = 1$ to $N-1$
2. Set PTR: =1
3. Repeat while $PTR \leq N-K$
 - (a) If $DATA[PTR] > DATA[PTR+1]$, then:
Interchange $DATA[PTR]$ and $DATA[PTR+1]$ [End of if structure]
 - (b) Set $PTR: = PTR+1$
[End of inner loop][End of Step 1 Outer loop]
4. Exit



Data Structures

❑ Complexity of the bubble sort algorithm

- The time for a sorting algorithm is measured in terms of the number of comparisons. The number $f(n)$ of comparisons in the bubble sort is easily computed. Specifically, there are $n-1$ comparisons during first pass, which places the largest element in the last position, there are $n-2$ comparisons in the second pass, which places the second largest element in the next to last position, and so on. Thus,

$$f(n) = (n-1) + (n-2) + \dots + 2 + 1 = n(n-1)/2 = n^2/2 + O(n) = O(n^2)$$

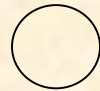
- In other words, the time required to execute bubble sort algorithm is proportional to n^2 , where n is the number of input items.



Data Structures

- ❑ **Solved Problem 4.9:** Using the bubble sort algorithm, find the number C of comparisons and the number D of interchanges which alphabetize the $n=6$ letters in PEOPLE.

Pass 1: PEO P L E, E P O P L E, E O P P L E
E O P P L E, E O P L P E, E O P L E P



Pass 2: E O P L E P, E O P L E P, E O P L E P
E O L P E P, E O L E P P



Pass 3: E O L E P P, E O L E P P, E L O E P P
E L E O P P



Pass 4: E L E O P P, E L E O P P, E E L O P P

Pass 5: E E L O P P, E E L O P P



Data Structures

- ❑ **Solved Problem 4.9:** Using the bubble sort algorithm, find the number C of comparisons and the number D of interchanges which alphabetize the $n=6$ letters in PEOPLE.

Since $n=6$, the number of comparisons, $C = 5+4+3+2+1=15$ and the number of interchanges, $D = 9$.



Data Structures

❑ **Solved Problem 4.11:** Suppose multidimensional arrays A and B are declared using

$A(-2:2, 2:22)$ and $B(1:8, -5:5, -10:5)$

- (a) Find the length of each dimension and the number of elements in A and B.
- (b) Consider the element $B[3, 3, 3]$ in B . Find the effective indices E1, E2, E3 and the address of the element, assuming $\text{Base}(B)=400$ and there are $w=4$ words per memory location.

Solution:

(a) $\text{Length} = \text{UB} - \text{LB} + 1$

Hence, the lengths of the dimensions of A are:

$$L_1 = 2 - (-2) + 1 = 5 \text{ and } L_2 = 22 - 2 + 1 = 21$$

Accordingly, A has $5 \cdot 21 = 105$ elements.

The lengths of the dimensions of B are:

$$L_1 = 8 - 1 + 1 = 8, \quad L_2 = 5 - (-5) + 1 = 11, \text{ and } L_3 = 5 - (-10) + 1 = 16$$

Therefore, B has $8 \cdot 11 \cdot 16 = 1408$ elements.



Data Structures

❑ **Solved Problem 4.11:** Suppose multidimensional arrays A and B are declared using

$A(-2:2, 2:22)$ and $B(1:8, -5:5, -10:5)$

(b) Consider the element $B[3, 3, 3]$ in B . Find the effective indices E_1, E_2, E_3 and the address of the element, assuming $\text{Base}(B)=400$ and there are $w=4$ words per memory location.

Solution:

(b) The effective index E_i is obtained from $E_i = k_i - LB$, where k_i is the given index and LB is the lower bound . Hence,

$$E_1 = 3 - 1 = 2 \quad E_2 = 3 - (-5) = 8 \quad E_3 = 3 - (-10) = 13$$

The address depends on whether the programming language stores B in row-major order or column-major order. Assuming B is the stored in column–major order, we get:

$$\begin{aligned} E_3 L_2 &= 13 \cdot 11 = 143 & E_3 L_2 + E_2 &= 143 + 8 = 151 \\ (E_3 L_2 + E_2) L_1 &= 151 \cdot 8 = 1208 & (E_3 L_2 + E_2) L_1 + E_1 &= 1208 + 2 = 1210 \end{aligned}$$

Therefore, $\text{LOC}(B[3,3,3]) = 400 + 4(1210) = 400 + 4840 = 5240$